Gốc Việt Publisher — complete upload script

Internal command-line tool used only by the owner of the YouTube channel "Gốc Việt" (channel ID UC-KJsMzxn_kixzKqfmRFugw) on a personal computer. Two files, complete and unmodified: **yt_auth.py** (OAuth authorization and revocation) and **publish_youtube.py** (upload).

Requirements: Python 3.9+, `pip install google-api-python-client google-auth-oauthlib requests`. Not included: the private `.secrets/` folder (OAuth client file and token) and `.secrets/youtube.env` (non-secret flags `AUDITED`, `EXPECTED_CHANNEL_ID`). Code comments and console messages are in Vietnamese, the owner's language.

Usage: `python yt_auth.py` (authorize once; `--reauth` to show the consent screen again; `--revoke` to revoke the token with Google and delete it) → `python publish_youtube.py videos/<id> --at "YYYY-MM-DD HH:MM"` (dry run) → add `--yes` to upload.

What publish_youtube.py does:

1. Reads the title, description (chapters + cited sources), tags and English localization from the video's `publish.md`.
2. Dry run by default: prints every setting; uploads only with `--yes`.
3. Refuses to upload if the video was already uploaded (local record), if the authorized channel is not "Gốc Việt", or if the project is not yet audited (unless `--test`).
4. `videos.insert` (resumable): privacyStatus = private + publishAt (scheduled), categoryId 27 (Education), selfDeclaredMadeForKids = false, containsSyntheticMedia = true, license = youtube, embeddable, notifySubscribers, defaultLanguage = vi, English localization.
5. `thumbnails.set` (1280×720 JPG), `captions.insert` (English SRT), adds the video to a playlist (`playlists.list/insert`, `playlistItems.insert`).
6. Records only the file name and time locally to prevent duplicate uploads; no video IDs or other YouTube API data are stored (the video URL is only printed on screen).
7. If the token can no longer be refreshed (e.g. access was revoked), both scripts delete it from the computer.

yt_auth.py

```
"""Kết nối kênh YouTube Gốc Việt (OAuth, làm 1 lần).

  tools/.venv/bin/python tools/yt_auth.py            # kết nối (bỏ qua nếu mã còn dùng được)
  tools/.venv/bin/python tools/yt_auth.py --reauth   # mở lại màn hình đồng ý của Google, cấp mã mới
  tools/.venv/bin/python tools/yt_auth.py --revoke   # thu hồi mã với Google ngay + xóa mã trên máy

Cần: .secrets/youtube_client.json (OAuth client loại "Desktop app" tải từ Google Cloud Console).
Trình duyệt sẽ mở ra: đăng nhập tài khoản Google quản lý kênh → chọn kênh Gốc Việt → cho phép.
Mã lưu vào .secrets/youtube_token.json (quyền 600). Không in mã ra màn hình.
Mã bị thu hồi / không làm mới được → tự xóa (chính sách YouTube API III.D.2).
"""
import argparse
import json
import os
import sys
from pathlib import Path

import requests
from google.auth.exceptions import RefreshError
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

ROOT = Path(__file__).resolve().parent.parent
SECRETS = ROOT / ".secrets"
```

```
CLIENT = SECRETS / "youtube_client.json"
TOKEN = SECRETS / "youtube_token.json"
SCOPES = ["https://www.googleapis.com/auth/youtube.upload", "https://www.googleapis.com/auth/youtube.force-ssl"]


def expected_channel():
    p = SECRETS / "youtube.env"
    if p.exists():
        for line in p.read_text(encoding="utf-8").splitlines():
            if line.startswith("EXPECTED_CHANNEL_ID="):
                return line.split("=", 1)[1].strip() or None
    return None


def load_creds():
    if not TOKEN.exists():
        return None
    creds = Credentials.from_authorized_user_file(str(TOKEN), SCOPES)
    if creds and creds.expired and creds.refresh_token:
        try:
            creds.refresh(Request())
        except RefreshError:
            TOKEN.unlink(missing_ok=True)
            print("Mã cũ không còn hiệu lực (có thể đã bị thu hồi) — đã xóa khỏi máy, cấp mã mới.")
            return None
        save(creds)
    return creds


def save(creds):
    TOKEN.write_text(creds.to_json())
    os.chmod(TOKEN, 0o600)


def revoke():
    if not TOKEN.exists():
        sys.exit("Không có mã nào trên máy để thu hồi.")
    d = json.loads(TOKEN.read_text())
    r = requests.post("https://oauth2.googleapis.com/revoke", params={"token": d.get("refresh_token") or d.get("token")},
                      headers={"content-type": "application/x-www-form-urlencoded"}, timeout=30)
    TOKEN.unlink(missing_ok=True)
    print(f"✓ Đã thu hồi quyền với Google ({'thành công' if r.status_code == 200 else f'HTTP {r.status_code} — mã có thể đã hết hiệu lực'})")
    print(f"✓ Đã xóa {TOKEN.relative_to(ROOT)}")


def main():
    ap = argparse.ArgumentParser(description="Kết nối / thu hồi quyền kênh YouTube Gốc Việt")
    ap.add_argument("--reauth", action="store_true", help="mở lại màn hình đồng ý của Google và cấp mã mới")
    ap.add_argument("--revoke", action="store_true", help="thu hồi mã với Google ngay và xóa mã trên máy")
    a = ap.parse_args()
    if a.revoke:
        revoke()
        return
    if not CLIENT.exists():
        sys.exit(f"Chưa có {CLIENT.relative_to(ROOT)} — tải file OAuth client (Desktop app) từ Google Cloud Console vào đây.")
```

```
    creds = None if a.reauth else load_creds()
    if not creds or not creds.valid:
        flow = InstalledAppFlow.from_client_secrets_file(str(CLIENT), SCOPES)
        print("Đang mở trình duyệt để bạn đăng nhập Google và cho phép…")
        creds = flow.run_local_server(port=0, prompt="consent", access_type="offline", open_browser=True,
                                      authorization_prompt_message="Nếu trình duyệt không tự mở, mở link sau:\n{url}",
                                      success_message="Đã kết nối Gốc Việt Publisher. Bạn có thể đóng tab này.")
        save(creds)
    yt = build("youtube", "v3", credentials=creds, cache_discovery=False)
    ch = yt.channels().list(part="snippet,status", mine=True).execute().get("items", [])
    if not ch:
        sys.exit("Tài khoản này chưa có kênh YouTube (hoặc chưa chọn đúng kênh khi cấp quyền).")
    c = ch[0]
    exp = expected_channel()
    if exp and c["id"] != exp:
        TOKEN.unlink(missing_ok=True)
        sys.exit(f"✗ Bạn đã chọn kênh “{c['snippet']['title']}” (ID {c['id']}), không phải kênh Gốc Việt (ID {exp}).\n"
                 "  Đã xóa mã vừa cấp. Chạy lại và ở màn hình “Chọn tài khoản/kênh”, chọn đúng kênh Gốc Việt.")
    print(f"✓ Đã kết nối kênh: {c['snippet']['title']} (ID {c['id']})")
    print(f"✓ Quyền: tải video, quản lý thumbnail/phụ đề/danh sách phát · mã làm mới: {'có' if creds.refresh_token else 'KHÔNG'}")
    print(f"✓ Lưu tại {TOKEN.relative_to(ROOT)} (chỉ máy này đọc được)")
    if not creds.refresh_token:
        print("⚠ Không có mã làm mới — kiểm tra màn hình đồng ý OAuth đã ở chế độ 'In production' chưa.")


if __name__ == "__main__":
    main()
```

publish_youtube.py

```
"""Đăng video lên YouTube Gốc Việt theo cài đặt chuẩn (CLAUDE.md §6b).

  # chạy thử (mặc định — không tải gì)
  tools/.venv/bin/python tools/publish_youtube.py videos/<id> --at "2026-10-23 19:30"
  # tải thật + lên lịch (chỉ khi dự án đã qua kiểm duyệt, hoặc --test để thử ở chế độ riêng tư)
  tools/.venv/bin/python tools/publish_youtube.py videos/<id> --at "2026-10-23 19:30" --yes

Lấy tiêu đề / mô tả / thẻ / bản tiếng Anh từ mục "## YouTube" trong publish.md.
Cài đặt: Giáo dục (27), không dành cho trẻ em, containsSyntheticMedia, giấy phép YouTube, cho nhúng,
thông báo người đăng ký, ngôn ngữ vi, thumbnail, phụ đề EN, danh sách phát.
Dự án Google CHƯA kiểm duyệt → video bị khóa riêng tư dù đặt lịch (giới hạn của YouTube).
"""
import argparse
import json
import re
import sys
import time
from datetime import datetime, timezone
from pathlib import Path
from zoneinfo import ZoneInfo

ROOT = Path(__file__).resolve().parent.parent
SECRETS = ROOT / ".secrets"
```

```
TZ = ZoneInfo("Asia/Ho_Chi_Minh")
PLAYLIST_DEFAULT = "Chuyện lịch sử Việt Nam"


def block_after(text, label):
    m = re.search(re.escape(label) + r"[^\n]*\n```[^\n]*\n(.*?)```", text, flags=re.S)
    return m.group(1).strip() if m else None


def meta_from_publish(vdir):
    t = (vdir / "publish.md").read_text(encoding="utf-8")
    m = re.search(r"^## YouTube[^\n]*\n(.*?)(?=^## |\Z)", t, flags=re.S | re.M)
    if not m:
        sys.exit("publish.md không có mục '## YouTube'.")
    sec = m.group(1)
    title = block_after(sec, "**Tiêu đề:**")
    desc = block_after(sec, "**Mô tả:**")
    tags = block_after(sec, "**Tags:**")
    en_t = re.search(r"- Title: `([^`]+)`", sec)
    en_d = re.search(r"- Description: `([^`]+)`", sec)
    if not (title and desc):
        sys.exit("Mục YouTube trong publish.md thiếu khối ``` cho Tiêu đề hoặc Mô tả.")
    if "{CHAPTERS}" in desc:
        sys.exit("Mô tả còn {CHAPTERS} — điền mốc chương trước.")
    return {"title": title, "description": desc,
            "tags": [x.strip() for x in (tags or "").split(",") if x.strip()],
            "en_title": en_t.group(1) if en_t else None, "en_desc": en_d.group(1) if en_d else None}


def audited():
    p = SECRETS / "youtube.env"
    return p.exists() and re.search(r"^AUDITED=1\s*$", p.read_text(), flags=re.M) is not None


def main():
    ap = argparse.ArgumentParser(description="Đăng video lên YouTube Gốc Việt")
    ap.add_argument("video_dir")
    ap.add_argument("--file", help="mp4 (mặc định output/*_16x9.mp4)")
    ap.add_argument("--at", help='giờ công khai (giờ VN) "YYYY-MM-DD HH:MM" — bỏ trống = để riêng tư')
    ap.add_argument("--playlist", default=PLAYLIST_DEFAULT)
    ap.add_argument("--test", action="store_true", help="cho phép tải khi dự án chưa kiểm duyệt (video sẽ bị khóa riêng tư)")
    ap.add_argument("--yes", action="store_true", help="tải thật (không có cờ này = chạy thử)")
    a = ap.parse_args()

    vdir = (ROOT / a.video_dir).resolve()
    vid_name = vdir.name
    f = Path(a.file) if a.file else next(iter(sorted((vdir / "output").glob("*_16x9.mp4"))), None)
    if not f or not f.exists():
        sys.exit("Không thấy file video 16:9.")
    thumb = vdir / "output" / "thumbnail_youtube_1280x720.jpg"
    srt = next(iter(sorted((vdir / "output").glob("*_en.srt"))), None)
    meta = meta_from_publish(vdir)
    if len(meta["title"]) > 100:
        sys.exit("Tiêu đề YouTube dài hơn 100 ký tự.")
```

```
publish_at = None
if a.at:
    dt = datetime.strptime(a.at, "%Y-%m-%d %H:%M").replace(tzinfo=TZ)
    if dt.timestamp() < time.time() + 15 * 60:
        sys.exit("Giờ công khai phải cách bây giờ ít nhất 15 phút.")
    publish_at = dt.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S.000Z")

log = vdir / "publish_log.json"
entries = json.loads(log.read_text()) if log.exists() else []
if any(e.get("platform") == "youtube" and e.get("state") != "test" for e in entries):
    sys.exit("Video này đã được tải lên YouTube trước đó (xem publish_log.json) — không tải trùng.")

print("—" * 60)
print(f"File:      {f.name}")
print(f"Tiêu đề:   {meta['title']}  ({len(meta['title'])} ký tự)")
print(f"Thẻ:       {', '.join(meta['tags'])[:160]}")
print(f"Lịch:      {'công khai lúc ' + a.at + ' (giờ VN)' if publish_at else 'riêng tư (chưa lên lịch)'}")
print(f"Thumbnail: {thumb.name if thumb.exists() else '(thiếu!)'} · Phụ đề EN: {srt.name if srt else '(thiếu)'} · Danh sách phát: {a.playlist}")
print(f"Bản EN:    {'có' if meta['en_title'] else 'không'} · Cài đặt: Giáo dục · không dành cho trẻ em · nội dung tổng hợp · giấy phép YouTube · cho nhúng")
print(f"Kiểm duyệt dự án: {'ĐÃ DUYỆT' if audited() else 'CHƯA — video sẽ bị khóa riêng tư'}")
print("—" * 60)
if not a.yes:
    print("CHẠY THỬ — chưa tải gì. Thêm --yes để tải thật.")
    return
if not audited() and not a.test:
    sys.exit("Dự án chưa được Google kiểm duyệt → video tải qua API sẽ bị khóa riêng tư. Dùng --test để thử đường truyền, hoặc đăng tay.")

from google.auth.exceptions import RefreshError
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload
from googleapiclient.errors import HttpError

tok = SECRETS / "youtube_token.json"
if not tok.exists():
    sys.exit("Chưa kết nối YouTube — chạy tools/yt_auth.py.")
creds = Credentials.from_authorized_user_file(str(tok))
if creds.expired and creds.refresh_token:
    try:
        creds.refresh(Request())
    except RefreshError:
        tok.unlink(missing_ok=True)
        sys.exit("Mã YouTube không còn hiệu lực (có thể đã bị thu hồi) — đã xóa khỏi máy. Chạy lại tools/yt_auth.py.")
    tok.write_text(creds.to_json())
yt = build("youtube", "v3", credentials=creds, cache_discovery=False)
exp = re.search(r"^EXPECTED_CHANNEL_ID=(\S+)", (SECRETS / "youtube.env").read_text(), flags=re.M)
me = yt.channels().list(part="snippet", mine=True).execute().get("items", [])
if not me or (exp and me[0]["id"] != exp.group(1)):
    sys.exit(f"✗ Mã đang nối với kênh {me[0]['snippet']['title'] if me else '?'} — không phải kênh Gốc Việt. Chạy lại tools/yt_auth.py.")
print(f"✓ Kênh: {me[0]['snippet']['title']} (ID {me[0]['id']})")

body = {
    "snippet": {"title": meta["title"], "description": meta["description"], "tags": meta["tags"],
                "categoryId": "27", "defaultLanguage": "vi", "defaultAudioLanguage": "vi"},
```

```
        "status": {"privacyStatus": "private", "selfDeclaredMadeForKids": False, "license": "youtube",
                   "embeddable": True, "publicStatsViewable": True, "containsSyntheticMedia": True},
    }
    if publish_at:
        body["status"]["publishAt"] = publish_at
    parts = "snippet,status"
    if meta["en_title"]:
        body["localizations"] = {"en": {"title": meta["en_title"][:100], "description": meta["en_desc"] or ""}}
        parts += ",localizations"
    try:
        req = yt.videos().insert(part=parts, body=body, notifySubscribers=True,
                                 media_body=MediaFileUpload(str(f), mimetype="video/mp4", chunksize=8 * 1024 * 1024, resumable=True))
        resp = None
        while resp is None:
            st, resp = req.next_chunk()
            if st:
                print(f"  … tải lên {st.progress() * 100:.0f}%")
        vid = resp["id"]
        print(f"✓ Đã tải lên: https://youtu.be/{vid}")
        if thumb.exists():
            yt.thumbnails().set(videoId=vid, media_body=MediaFileUpload(str(thumb), mimetype="image/jpeg")).execute()
            print("✓ Đã đặt thumbnail")
        if srt:
            yt.captions().insert(part="snippet", body={"snippet": {"videoId": vid, "language": "en", "name": "English"}},
                                 media_body=MediaFileUpload(str(srt), mimetype="application/octet-stream")).execute()
            print("✓ Đã tải phụ đề tiếng Anh")
        pls = yt.playlists().list(part="snippet", mine=True, maxResults=50).execute().get("items", [])
        pl = next((p for p in pls if p["snippet"]["title"] == a.playlist), None)
        if not pl:
            pl = yt.playlists().insert(part="snippet,status", body={"snippet": {"title": a.playlist, "defaultLanguage": "vi"},
                                                                    "status": {"privacyStatus": "public"}}).execute()
        yt.playlistItems().insert(part="snippet", body={"snippet": {"playlistId": pl["id"],
                                  "resourceId": {"kind": "youtube#video", "videoId": vid}}}).execute()
        print(f"✓ Đã thêm vào danh sách phát: {a.playlist}")
    except HttpError as e:
        sys.exit(f"✗ YouTube báo lỗi: {e.status_code} {e.reason}")
    # Không lưu ID/link video: chính sách YouTube API chỉ cho lưu dữ liệu API tối đa 30 ngày (III.E.4) —
    # nhật ký chỉ ghi tên file + thời gian để chống tải trùng; link chỉ in ra màn hình.
    entries.append({"ts": int(time.time()), "platform": "youtube", "state": "scheduled" if publish_at else ("test" if a.test else "private"),
                    "scheduled_for": publish_at, "file": f.name})
    log.write_text(json.dumps(entries, ensure_ascii=False, indent=1))
    print(f"✓ Đã ghi nhật ký: {log.relative_to(ROOT)}")
    print("→ Việc tay: ghim bình luận sau khi video công khai; thêm màn hình kết thúc trong Studio.")


if __name__ == "__main__":
    main()
```